JAVA CORE CONCEPTS

Operators in Java

Mathematical calculations, logical decisions, bit manipulation, and operator precedence

Operators are special symbols used to perform operations on variables and values.

```
int solvedWeek1 = 40;
int solvedWeek2 = 35;
int totalSolved = solvedWeek1 + solvedWeek2; // '+' is the addition operator
```

In Java, operators are fundamentally used across mathematical calculations, decision making, loop conditions, bit manipulation, and real-world logic building. Mastery of operators is critical for Data Structures & Algorithms (DSA), Competitive Programming, Backend Development, and Technical Interviews.

1. Types of Operators in Java

Java classifies operators into the following primary categories:

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Unary Operators
6. Bitwise Operators
7. Other Special Operators (`instanceof` , Ternary)

2. Arithmetic Operators

Arithmetic operators perform basic or complex mathematical calculations.

Operator	Meaning	Example Expression
`+`	Addition	`25 + 15 = 40`
`-`	Subtraction	`25 - 15 = 10`
`*`	Multiplication	`25 * 4 = 100`
`/`	Division	`25 / 7 = 3` (Truncates decimal)
`%`	Modulo (Remainder)	`25 % 7 = 4`

Code Demonstration

```java
public class Main {
    public static void main(String[] args) {
        int solvedThisWeek = 25;
        int solvedLastWeek = 15;

        int total = solvedThisWeek + solvedLastWeek;      // 40
        int difference = solvedThisWeek - solvedLastWeek; // 10
        int projected = solvedThisWeek * 4;               // 100
        int average = solvedThisWeek / 7;                 // 3
        int remainder = solvedThisWeek % 7;               // 4

        System.out.println(total);
        System.out.println(difference);
        System.out.println(projected);
        System.out.println(average);
        System.out.println(remainder);
    }
}
```

> ⚠️ **Integer Division Truncation**
>
> Since both operands in `solvedThisWeek / 7` are integers, Java performs integer division. The fractional component (`3.57`) is discarded entirely, yielding `3`.

Note on Modulo (`%`): Heavy application in even/odd checks (`num % 2 == 0`), circular array indexing (`(i + 1) % N`), and digit extraction problems.

3. Assignment Operators

Assignment operators assign evaluated values to variables. They include basic and compound variants.

Operator	Meaning	Equivalent Compound Form
`=`	Simple Assignment	`x = y`
`+=`	Add and assign	`x = x + y`
`-=`	Subtract and assign	`x = x - y`
`*=`	Multiply and assign	`x = x * y`
`/=`	Divide and assign	`x = x / y`
`%=`	Modulo and assign	`x = x % y`

Code Example & Step-by-Step Execution

```java
public class Main {
    public static void main(String[] args) {
        int ratingPoints = 100;
        ratingPoints += 20;  // 100 + 20 = 120
        ratingPoints -= 10;  // 120 - 10 = 110
        ratingPoints *= 2;   // 110 * 2  = 220
        ratingPoints /= 4;   // 220 / 4  = 55
        ratingPoints %= 30;  // 55 % 30  = 25

        System.out.println(ratingPoints); // Prints 25
    }
}
```

4. Relational Operators

Relational operators compare two values and evaluate to a `boolean` result (`true` or `false`).

Operator	Meaning	Example (45 vs 50)	Result
`==`	Equal to	`45 == 50`	`false`
`!=`	Not equal to	`45 != 50`	`true`
`>`	Greater than	`45 > 50`	`false`
`<`	Less than	`45 < 50`	`true`
`>=`	Greater than or equal to	`45 >= 50`	`false`
`<=`	Less than or equal to	`45 <= 50`	`true`

5. Logical Operators

Logical operators combine multiple boolean expressions to determine complex decision pathways.

Operator Descriptions:

- `&&` **(Logical AND):** Returns `true` only if both operands are `true` . Short-circuits if the left side is `false` .
- `||` **(Logical OR):** Returns `true` if at least one operand is `true` . Short-circuits if the left side is `true` .
- `!` **(Logical NOT):** Inverts/reverses the boolean truth value.

```java
public class Main {
    public static void main(String[] args) {
        boolean completedDSA = true;
        boolean completedCore = false;

        System.out.println(completedDSA && completedCore); // false
        System.out.println(completedDSA || completedCore); // true
        System.out.println(!completedCore);                // true
    }
}
```

6. Unary Operators

Unary operators operate on a single operand to modify its sign, increment, decrement, or negate its logical state.

Operator	Name	Description
`+`	Unary Plus	Indicates positive value (default)
`-`	Unary Minus	Negates an expression value
`++`	Increment	Increments value by 1
`--`	Decrement	Decrements value by 1
`!`	Logical Complement	Inverts boolean value

Prefix vs Postfix Mechanics

A classic interview topic centers on the execution difference between Prefix (`++var`) and Postfix (`var++`):

```
public class Main {
    public static void main(String[] args) {
        int activeUsers = 100;

        int prefix = ++activeUsers; // Increment first (101), then assign -> prefix = 101
        int postfix = activeUsers++; // Assign first (101), then increment -> postfix = 101

        System.out.println(prefix);      // 101
        System.out.println(postfix);     // 101
        System.out.println(activeUsers); // 102 (Final value)
    }
}
```

7. Bitwise Operators

Bitwise operators act directly on individual binary bits of integer data types (`byte` , `short` , `int` , `long`).

Operator	Meaning	Operation Rule
`&`	Bitwise AND	1 if both bits are 1
`|`	Bitwise OR	1 if at least one bit is 1
`^`	Bitwise XOR	1 if bits are different
`~`	Bitwise Complement	Flips all bits (0→1, 1→0)
`<<`	Left Shift	Shifts bits left, fills right with 0 (Multiplies by 2^n)
`>>`	Right Shift	Shifts bits right with sign extension (Divides by 2^n)
`>>>`	Unsigned Right Shift	Shifts bits right, fills left with 0s regardless of sign

Bitwise Calculation Example

Given `x = 6` (`00000110` in binary) and `y = 3` (`00000011` in binary):

```
int x = 6;  // 00000110
int y = 3;  // 00000011

System.out.println(x & y);  // 00000010 -> 2
System.out.println(x | y);  // 00000111 -> 7
System.out.println(x ^ y);  // 00000101 -> 5
System.out.println(~x);     // Bitwise flip -> -7
System.out.println(x << 1); // 6 * 2^1 -> 12
System.out.println(x >> 1); // 6 / 2^1 -> 3
```

8. Special Operators: instanceof & Ternary

instanceof Operator

Checks whether an object instance belongs to a specified class or subtype reference.

```
String track = "CodeHelp ONE";
boolean result = track instanceof String; // Evaluates to true
```

Ternary Operator (`? :`)

A concise inline shorthand for simple `if-else` conditional expressions.

```
int solvedProblems = 320;
String level = (solvedProblems >= 300) ? "Advanced" : "Intermediate";
System.out.println(level); // Outputs "Advanced"
```

9. Operator Precedence Hierarchy

Operators are evaluated in order of strict precedence. Understanding this order prevents logical execution bugs.

Precedence Level	Operators	Associativity
1 (Highest)	Postfix: `expr++` `expr--`	Left-to-Right
2	Unary: `++expr` `--expr` `+` `-` `~` `!`	Right-to-Left
3	Multiplicative: `*` `/` `%`	Left-to-Right
4	Additive: `+` `-`	Left-to-Right
5	Shift: `<<` `>>` `>>>`	Left-to-Right
6	Relational: `<` `>` `<=` `>=` `instanceof`	Left-to-Right
7	Equality: `==` `!=`	Left-to-Right
8	Bitwise AND: `&`	Left-to-Right
9	Bitwise XOR: `^`	Left-to-Right
10	Bitwise OR: `|`	Left-to-Right
11	Logical AND: `&&`	Left-to-Right
12	Logical OR: `||`	Left-to-Right
13	Ternary: `? :`	Right-to-Left
14 (Lowest)	Assignment: `=` `+=` `-=` `*=` `/=` `%=`	Right-to-Left

```java
int result = 10 + 5 * 2; // Evaluates as 10 + (5 * 2) = 20 due to multiplication precedence
```

💡 Why Operator Mastery Matters for Coding Rounds

Most candidate rejections or bug failures in online assessment rounds stem from:

- **Integer Division Pitfalls:** Forgetting that `a / b` drops fractional results when both are integers.
- **Prefix vs Postfix Confusion:** Incorrect state assumptions in complex loops.
- **Precedence Assumptions:** Omitting necessary parentheses in complex logical conditions.
- **Bit Manipulation Efficiency:** Missing fast $O(1)$ bit tricks in Competitive Programming.